

PARIMIT

BOUNDED AUTHORITY FOR AGENT PAYMENTS

Parimit: A Proposal-Only Authorization Boundary for Agentic Payments

Technical White Paper v0.2

Implementation, identity, evidence, and local AiNxt compatibility study

IMPLEMENTATION-ALIGNED TECHNICAL ALPHA

Parimit Contributors

29 September 2026

Open source - MIT License

Proposal and evidence only. No UPI, bank, PSP, wallet, or live payment connection.

Contents

This paper distinguishes implementation, automated evidence, maintainer-observed live evidence, human acceptance, and external acceptance.

Abstract 3

Executive summary 3

1. Problem: authority collapses too easily 5

2. Contribution and claim boundary 5

3. Architecture 6

4. Authority and lifecycle model 7

5. Evidence and integrity 8

6. Identity and the Keycloak pilot 8

7. AiNxt compatibility reference 9

8. Evaluation 10

9. Reproducibility snapshot 11

10. Threat model and residual risk 13

11. Roadmap and decision gates 13

12. Conclusion 14

References 15

Appendix A - Public capability inventory 15

Appendix B - Evaluation boundary 16

Appendix C - Independent project statement 16

Abstract

AI agents can discover products, prepare transactions, and generate structured payment requests. The risky step is allowing the same agent to approve, sign, dispatch, or retry the action. Parimit explores a narrow response: place a deterministic, non-dispatchable authorization boundary between an untrusted agent and any future payment executor.

Parimit allows an authenticated agent to create, inspect, simulate, and cancel a payment proposal. It validates a closed schema, applies deterministic policy, requires one or two decisions from distinct authenticated approver subjects, binds those decisions to the exact proposal, and records tamper-evident local audit evidence. The intended workflow assigns those approver subjects to people; the runtime proves subject and role separation, not biological humanness. After complete approval it can issue a short-lived, audience-bound Ed25519 Authorization Envelope whose signed claims explicitly state `execution_authorized: false` and `moves_money: false`. No shipped interface can convert that evidence into a payment instruction.

This paper describes the implemented alpha, the local Keycloak identity pilot, and an optional trusted-sidecar adapter for NPCI's public AiNxt OS HTTP/SSE contract. A maintainer-observed local run combined an allowlisted model tag with a separately recorded local model digest, a reviewed AiNxt build, real Keycloak-signed workload identity, and Parimit. All 16 live compatibility checks and all 12 workload-identity checks passed. The allowed synthetic coffee fixture ended at `AWAITING_APPROVAL` with zero approvals; the higher-risk mobility fixture was denied before creation. These results demonstrate a bounded proposal path. They do not demonstrate human acceptance, UPI connectivity, payment execution, NPCI acceptance, or production readiness.

Executive summary

Parimit's thesis is simple:

A machine may prepare a payment proposal without receiving the authority to approve or execute it.

The implemented system separates five concerns:

- 1 **Proposal:** an agent describes an intended payment using a strict, validated structure.
- 2 **Policy:** ordinary code evaluates amount, payee, velocity, expiry, and approval requirements.
- 3 **Human decision:** a separately authenticated reviewer approves or rejects the exact proposal digest.
- 4 **Evidence:** the service issues non-dispatchable receipts and, after full approval, a signed and audience-bound evidence envelope.
- 5 **Execution:** absent from the repository and outside Parimit's trust boundary.

The result is not a payment processor. It is a reference governance layer that can sit before a future, separately operated executor. The strongest current property is negative: the shipped system has no route, credential, provider SDK, or capability that can move funds.

Results at a glance

Evidence layer	Result	What it supports	What it does not support
Core automated suite	94 of 94 passed	Domain, policy, identity, evidence, HTTP, persistence, adapter, and adversarial behavior	Production security or regulatory acceptance
TypeScript SDK suite	6 of 6 passed	Role-shaped client behavior and safe endpoint use	Compatibility with every client environment
Boundary scanner	Passed	Selected source and capability invariants	Formal verification or whole-program data-flow proof
Keycloak workload smoke	12 of 12 passed	TLS/OIDC workload identity and role separation	Interactive human acceptance
Live AiNxt-Keycloak smoke	16 of 16 passed	Reviewed local model-to-proposal compatibility	Model-byte attestation, native AiNxt integration, UPI, or execution
Prior feature PR/main checks	Passed	CI, CodeQL, secret scan, dependency review, and pilot guardrails before the pin correction	Hosted checks for d9807d3 or independent penetration testing

The source, documentation, tests, and build workflows are open under the MIT License. This paper accompanies the [v0.1.0-alpha.4](#) public research alpha. Its live compatibility claims remain tied to the exact tested runtime snapshot identified in Section 9 rather than to an unqualified production claim.

1. Problem: authority collapses too easily

Agentic systems combine planning, external content, models, tools, memory, and retry logic. That composition creates risks that ordinary checkout interfaces do not solve on their own.

1.1 Authority collapse

If one agent can propose, approve, and execute a payment, policy is reduced to a prompt. Prompt injection, model error, compromised tools, or a confused deputy can become financial authority.

1.2 Intent substitution

A person may review one amount, payee, or purpose while a downstream component acts on another. Human confirmation has little value unless it is bound to an exact, immutable intent.

1.3 Identity ambiguity

Role labels are not identities. An agent, human reviewer, evidence consumer, and administrator must be cryptographically separated. A service account with an `approver` label would invalidate the human-control claim even if its token were correctly signed.

1.4 Uncertain outcomes and retries

An ambiguous provider response can trigger duplicate action. Parimit therefore treats `IN_DOUBT` as a freeze condition in its mock observation stream. The alpha exposes no reconciliation bypass.

1.5 Model output is data, not authorization

The AiNxt adapter treats model output as an untrusted draft. It accepts only closed, code-owned synthetic fixtures, validates exact JSON, simulates policy before persistence, and injects the authenticated agent identity on the trusted side. A fluent model response never becomes authority.

2. Contribution and claim boundary

The most defensible description of the project is:

Parimit is an independently implemented, open-source technical alpha for a proposal-only authorization and evidence boundary for AI agents.

2.1 Implemented contributions

- Proposal creation with agent-scoped idempotency and exact replay.
- Deterministic amount, daily exposure, payee, expiry, and approval policy.
- One- or two-approver semantics with distinct authenticated subjects.
- Exact-intent binding across proposal, policy snapshot, approvals, state, and evidence.
- HMAC-protected decision receipts and hash-linked audit events.
- Short-lived Ed25519 Authorization Envelope v1 evidence with public JWKS.
- Audience binding, exact expiry, one-time local consumption, signing-key history, and configuration/trust-domain binding.
- OIDC signature, issuer, audience, time, algorithm, key strength, token type, and lifetime checks.
- Server-enforced `agent`, `approver`, `consumer`, and `admin` capabilities.
- Browser demo, REST/OpenAPI surface, role-shaped TypeScript SDK, and a narrower proposal-only MCP server.
- Local TLS-enabled Keycloak identity profile with separate workload and human identities.
- Optional AiNxt trusted-sidecar reference adapter with strict draft validation and no approval or execution authority.

- PostgreSQL 14+ schema and transaction contract for a future storage port.

2.2 Claims explicitly excluded

Parimit is not NPCI-backed, UPI-enabled, bank-grade, PCI-compliant, regulator-approved, production-ready, formally verified, or a payment processor. It does not implement a live UPI, Reserve Pay, bank, PSP, wallet, or merchant checkout connector. It is not a native AiNxt tool, AiNxt MCP server, AtOM integration, or NPCI certification. It makes no market claim of being the first, only, or uniquely capable system.

3. Architecture

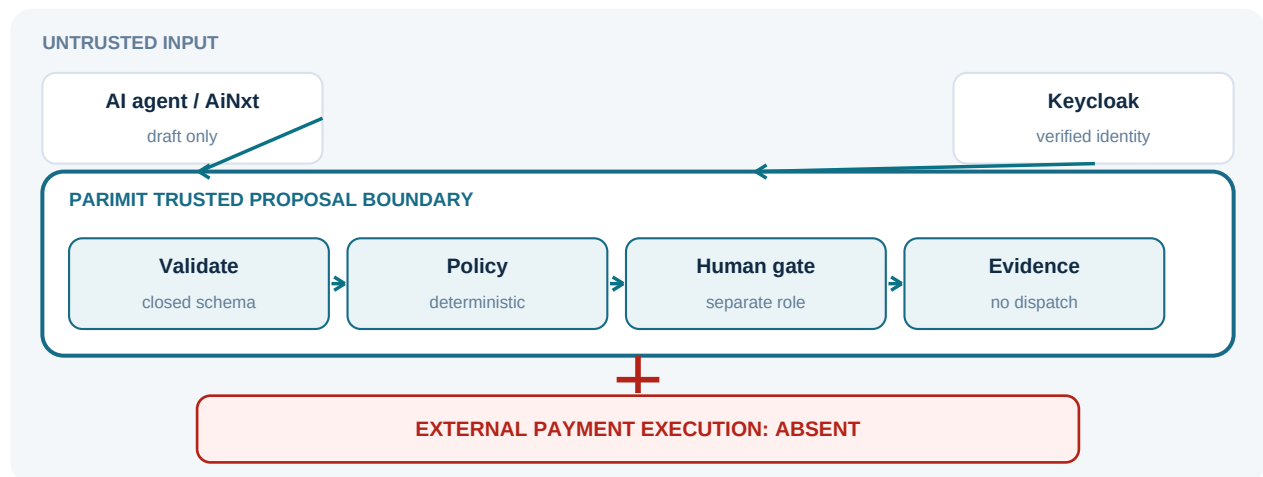


Figure 1. Parimit keeps external payment execution outside the shipped trust boundary.

The architecture is organized around a missing component: execution.

3.1 Proposal plane

An agent may call REST, the local MCP process, or the optional AiNxt adapter to simulate policy, create a proposal, inspect its own proposal, or cancel an eligible proposal. The MCP surface deliberately contains no approval, evidence-issuance, execution, payment, send, retry, or settlement tool.

3.2 Deterministic policy plane

Policy runs in ordinary TypeScript, not in a language model. It validates INR minor units, per-proposal limits, per-agent daily exposure, optional payee allowlists and blocklists, maximum expiry, and the number of required approver decisions. The policy snapshot and digest become part of later integrity verification.

3.3 Human decision plane

OIDC-protected HTTP routes separate the requesting agent from the reviewer and administrator. Each proposal is bound to the verified agent identity. Authenticated approver decisions bind the exact intent digest and state. Larger proposals can require two distinct authenticated subjects; the intended human workflow provisions those subjects to two different people.

The browser dashboard remains a local demonstration and is not an OIDC client. The controlled Keycloak pilot uses REST, the TypeScript SDK, and an interactive device flow for real human identities.

3.4 Evidence plane

After sufficient approval, Parimit can issue a public-key-verifiable evidence envelope. It records what was approved and the trust context in which that decision was made. It is deliberately non-dispatchable: it contains no rail credential, provider endpoint, bank instruction, or execution method, and it signs explicit false values for execution and money movement.

3.5 Mock observation plane

An administrator may record clearly labelled synthetic observations such as **SUCCEEDED**, **FAILED**, **REVERSED**, **DISPUTED**, or **IN_DOUBT**. These records do not represent real settlement. After **IN_DOUBT**, every later mock observation is rejected.

3.6 External execution plane

No external execution plane ships. Any future executor must be a separately packaged, separately authorized, separately audited component owned by a regulated participant. It must consume evidence once, revalidate the complete intent, own provider credentials and reconciliation, and remain unreachable from the Parimit core.

4. Authority and lifecycle model

4.1 Roles

Role	Permitted responsibility	Authority deliberately absent
agent	Simulate, create, inspect, and cancel its own proposals	Human decisions, evidence consumption, mock outcomes, execution
approver	Approve or reject an exact proposal	Creating agent-owned proposals, administration, execution
consumer	Verify and atomically consume an evidence envelope	Proposal browsing, approval, execution
admin	Read all proposals and audit; cancel eligible proposals; approve or reject; issue, verify, and consume evidence; record labelled mock outcomes	Creating as an agent, live provider action, money movement, or production-grade separation of duties

Every authenticated subject must map to exactly one internal role. Missing, unknown, or ambiguous mappings fail closed.

The **admin** role is deliberately broad for a local or synthetic pilot. It is not a production separation-of-duties design and must not be carried unchanged into a real payment deployment.

4.2 Lifecycle

A validated request is atomically evaluated and stored as either **POLICY_DENIED** or **AWAITING_APPROVAL**. An awaiting proposal may become **REJECTED**, **CANCELLED**, **EXPIRED**, or **AUTHORIZED_NO_DISPATCH**.

AUTHORIZED_NO_DISPATCH is intentionally precise. It means the local service has sufficient verified decisions for its policy. It does not mean a bank, wallet, PSP, or rail has authorized or received anything.

4.3 Exact intent and idempotency

The service stores integer INR minor units and rejects floating-point money. Proposal integrity binds immutable fields, policy state, trust configuration, and monotonic authorization state. Reusing an agent-scoped idempotency key with the same normalized request returns the original proposal; changing the request under the same key yields a conflict.

4.4 Fail-closed transitions

Expired intent, policy mismatch, role mismatch, ownership mismatch, clock rollback, audit inconsistency, decision tampering, trust-domain drift, signing key history changes, and replay conflict stop progress. The implementation does not silently downgrade those failures into warnings.

5. Evidence and integrity

Parimit uses related but distinct integrity mechanisms. They must not be described as one universal proof.

5.1 Intent and configuration digests

Versioned digests produced by Parimit's project-specific canonical JSON serializer bind proposal fields, policy configuration, tenant, trust settings, and authorization state. This is not a claim of RFC 8785/JCS conformance. Published test vectors make the encoding and signature inputs inspectable. Existing evidence is not silently rebound when configuration changes.

5.2 Decision receipts

Human decision records are HMAC-protected inside the deployment. This detects selected record changes while the receipt key and database root remain protected. HMAC is not public proof of a person's identity.

5.3 Authorization Envelope v1

After full approval, Parimit can issue a short-lived Ed25519/JWS envelope [7,9] that binds:

- the exact intent and policy/configuration digests;
- tenant, issuer, audience, identity trust domain, and key identifier;
- verified approver set and authorization state version;
- audit tip, issuance time, expiry, and envelope version; and
- explicit `execution_authorized: false` and `moves_money: false` claims.

Public verification keys are exposed through JWKS [8]. Verification rejects signature tampering, wrong audience, expiry, trust mismatch, removed historical keys, and inconsistent underlying evidence.

5.4 One-time consumption

A distinct `consumer` may atomically consume an envelope once inside the single SQLite service instance. The same consumer operation can replay idempotently; a different operation is rejected. Offline and distributed recipients still need their own durable replay ledger.

5.5 Local audit limits

Audit events are hash-linked and reconciled with proposals, decisions, envelopes, and mock observations. This is strong local consistency evidence, not an immutable external ledger. A privileged attacker controlling the host, database, process, and keys remains outside the alpha's protection model. No external monotonic anchor ships.

6. Identity and the Keycloak pilot

The local Keycloak profile exercises Parimit's real OIDC verifier over TLS. It is a controlled, single-machine acceptance environment using synthetic data.

6.1 Identity topology

External identity	Internal role	Identity type
<code>parimit-agent-workload</code>	<code>agent</code>	Service account
<code>pilot-reviewer</code>	<code>approver</code>	Interactive human account
<code>parimit-consumer-workload</code>	<code>consumer</code>	Service account
<code>pilot-admin</code>	<code>admin</code>	Interactive human account

The machine identities use separate client credentials. Reviewer and admin are not service accounts; they must use OAuth Device Authorization, replace generated temporary passwords, and configure TOTP at first login. Direct password grant, implicit flow, and browser login are disabled for the human CLI client [10].

6.2 Verification performed by Parimit

Parimit verifies token signature, exact issuer, exact audience, subject, issued-at and expiry times, bounded lifetime, allowed algorithm, optional token type, and an explicitly mapped top-level role claim. JWKS responses are size bounded and refreshed for key rotation. Spoofed local demo headers fail when OIDC mode is active [6].

6.3 Local-only limits

The profile uses Keycloak's development file database, one Keycloak instance, one Parimit instance, locally generated CA material, and loopback-published ports. It has no reverse proxy, high availability, external monitoring, or production backup system. Generated secrets and reports are ignored, excluded from the Docker build context, and created with restrictive permissions.

Automated workload smoke is not human acceptance. At this paper's snapshot, the reviewer/admin device-login, password-change, TOTP, and exact human confirmation exercise remains pending.

7. AiNxt compatibility reference

NPCI publishes AiNxt OS as a public foundation for building and operating AI applications and agents [4].

Parimit's adapter review is anchored to exact AiNxt source commit

454fb09cf1fff2bedb5aa3f4f1c391e4915f33dd.

The integration is complementary rather than embedded:



7.1 Adapter surface

The adapter declares only four operations:

- `parimit_simulate_proposal`
- `parimit_create_proposal`
- `parimit_get_proposal`
- `parimit_cancel_proposal`

It exposes no proposal listing, approval, rejection, evidence issuance, evidence consumption, mock observation, payment execution, UPI, bank, or PSP operation.

7.2 Preconditions

Before every operation, Parimit must report `PROPOSAL_ONLY`, no money movement, no UPI connection, no accepted live payment credentials, an empty execution route list, and one cryptographically verified `agent` identity. The adapter injects the verified actor identifier; the model cannot choose it.

7.3 Closed synthetic fixtures

The alpha accepts only the code-owned `coffee_order` and `mobility_pass` fixtures. Arbitrary free text and real payment identifiers are rejected. The model must return exact JSON matching the selected fixture. Extra fields, out-of-order or malformed SSE, a malformed or mid-turn-changing control-repo commit, oversized responses, or fixture drift stop before persistence. The combined runner additionally requires the exact operator-supplied expected control-repository commit.

7.4 Local model profile

The reviewed local path used Ollama `qwen3.5:4b` through a Parimit-owned, loopback-only OpenAI-compatible process. The process accepts only the exact model tag and expected route, caps request bodies, reconstructs headers, never forwards incoming `Authorization` or `Cookie`, and sends requests only to the loopback Ollama listener. The profile permits only AiNxt's `internal` data class.

The model tag is mutable. The local model ID was recorded separately in the component manifest [13], but the compatibility process did not attest model bytes at request time.

This compatibility process is not supplied by NPCI, AiNxt, or Ollama. Its role is narrow: adapt deterministic local inference controls for the reviewed runtime without adding payment authority.

7.5 What the integration proves

It proves that one reviewed AiNxt build can produce a strictly validated draft that a separately authenticated Parimit agent may simulate and convert into a proposal. It does not prove native AiNxt tool registration, a stable upstream plugin contract, AiNxt MCP integration, AtOM coordination, external identity acceptance, UPI connectivity, or NPCI approval.

8. Evaluation

8.1 Method

Evaluation used three layers:

- 1 deterministic automated tests over the domain, API, OIDC verifier, evidence, storage contract, SDK, adapter, and compatibility process;
- 2 a local Keycloak workload smoke exercising real signed tokens and TLS; and
- 3 a combined local run using an allowlisted model tag plus a separately recorded local model ID, reviewed AiNxt binary, real Keycloak agent token, adapter, and Parimit service.

All external identifiers and scenarios were synthetic. No human decision or payment execution was attempted by the automated runs.

8.2 Automated evidence

The core suite reported 94 passing tests and the TypeScript SDK suite reported 6 passing tests. The boundary check scanned selected source and integration files for prohibited authority surfaces and network behavior. GitHub checks on the prior merged feature [14] reported success for CI, CodeQL, dependency review, secret scanning, and the Keycloak pilot guardrails. The corrected commit `d9807d3` passed the same local 100-test and boundary suite; hosted checks for that correction remain a publication gate.

8.3 Keycloak workload smoke

The machine-only Keycloak smoke passed 12 of 12 checks. It verified loopback publication, clean-source inputs, canonical container rebuild, TLS discovery and JWKS, Parimit's non-execution declaration, signed audience-bound agent and consumer tokens, one-role identity mapping, missing-token and spoofed-header failure, role isolation, and the absence of approval or execution calls.

8.4 Live AiNxt-Keycloak proposal smoke

The corrected combined smoke passed 16 of 16 checks from a clean worktree at `d9807d3` [3]. In addition to the workload checks, it observed the AiNxt listener on loopback and applied Parimit's stricter policy requiring an exact 40-character lowercase full Git object identifier for the control-repository commit. Upstream defines the value as the control-repository commit but carries it as a string [5]. For this local study, the reviewed AiNxt checkout was deliberately designated as the control repository, so both revision values were `454fb09cf1fff2bedb5aa3f4f1c391e4915f33dd`. The smoke exercised two scenarios:

Scenario	Policy result	Persistence result	Final observed state
Synthetic mobility pass	Denied for amount, payee, and dual-approval conditions	No create request	No proposal
Synthetic coffee order	Preflight allowed	One create plus exact idempotent replay	<code>AWAITING_APPROVAL</code> , zero approvals

The report classification was `LIVE_AINXT_KEYCLOAK_PROPOSAL_SMOKE` and recorded `interactive_humans: false`, `approvals_attempted: 0`, and `payment_execution_capability: false`.

8.5 Interpretation

The evidence supports this bounded conclusion:

At the reviewed source and configuration snapshot, a local AiNxt model turn could be converted into one authenticated Parimit proposal while policy, identity, human-decision, and execution boundaries remained intact.

It does not support a conclusion about production availability, open-ended model inputs, adversarial network exposure, real human acceptance, regulatory compliance, bank integration, payment reliability, or money movement.

9. Reproducibility snapshot

The corrected live run used a clean detached Parimit worktree at commit `d9807d3c7f53c25464572f947c8c1daae690c355`. A publishable evidence summary [3] and non-secret component manifest [13] accompany this paper. Their values are maintainer-recorded identifiers, not independent attestation.

Component	Reviewed identifier
Parimit tested runtime commit	<code>d9807d3c7f53c25464572f947c8c1daae690c355</code>
AiNxt source-review commit	<code>454fb09cf1fff2bedb5aa3f4f1c391e4915f33dd</code>
AiNxt control-repository commit	<code>454fb09cf1fff2bedb5aa3f4f1c391e4915f33dd</code>
Local AiNxt binary SHA-256	<code>135a6522f66fd27985d2f9a71a3546ba629b80f512199bc40c7a7f9a8c3dce7c</code>
Ollama model	<code>qwen3.5:4b, 4.7B, Q4_K_M</code>
Maintainer-recorded local model ID	<code>2a654d98e6fba55d452b7043684e9b57a947e393bbffa62485a7aac05ee4eefd</code>
Parimit AiNxt profile SHA-256	<code>f91bfe85f02d3336a89509fc2afcb4ae6569ce9851e89700e25441f816186611</code>
Compatibility process SHA-256	<code>a2842cfb6c85145bb508c59256cde1ac0f8aa84e4326e2e3955581beef2b8f05</code>
Public component manifest SHA-256	<code>ed2103e2a304b6d195aeb056c2198189fb12bc56b32f47dd999b593c253710f4</code>
Public evidence summary SHA-256	<code>9f0e71d9b87992761b56d634acf0c5df314e901f1ce8c6722545afb363d76e8c</code>

Component	Reviewed identifier
Retained full redacted report SHA-256	79cd46b3f6438f68599835a08a5ed23f3fcc540bd04e9015ea574a44ab28d42d
Keycloak image	26.7.4@sha256:82a77884f3af238beab1e7afd63b5f530e1b5c0590bd7aa60b40a40463e29b2c
Node base image	24.21.0-bookworm-slim@sha256:0e0ff40c39bc087845bfb27465a0df4ea419520094bc35842ff83dd8cbe6f9b6

The AiNxt binary digest is a hash of one local build, not a claim that another toolchain will produce a byte-identical binary. The model, profile, process, and binary digests must be verified independently. The compatibility process enforces the mutable model tag, not the model content digest. AiNxt's reported `control_plane_sha` identifies the control-repository commit for the turn; it is not a substitute for binary, model, profile, or deployment attestation.

Generated credentials, private keys, human bootstrap details, and full runtime reports remain ignored. The published evidence summary deliberately omits run, intent, idempotency, actor, and credential data.

10. Threat model and residual risk

Threat	Implemented control	Residual limitation
Agent self-approval	Separate roles; no approval tool on agent surfaces	Human identity must still be operationally protected
Prompt or model manipulation	Closed fixtures, strict schema, exact matching, deterministic policy	Open-ended payment drafting is not supported
Intent substitution	Versioned digest and exact-decision binding	No external execution adapter exists to test downstream binding
Token leakage to model	Token stays inside trusted adapter and is not forwarded or reported	Host/process compromise remains out of scope
Replay	Agent idempotency and one-time local envelope consumption	Distributed recipients need their own durable ledger
Role confusion	OIDC verification and one-role mapping	Identity provisioning mistakes can invalidate governance claims
Audit modification	Hash links, HMAC/root checks, evidence reconciliation	No external append-only anchor
Ambiguous outcome retry	IN_DOUBT freezes later mock outcomes	Real provider reconciliation is not implemented
Exposed local pilot	Loopback-only ports and TLS	Docker egress is not an outbound firewall; profile is not production
Supply-chain drift	Source and image pins, component hashes, CI security checks	No reproducible-build proof or independent security certification

10.1 System limitations that remain material

- Interactive reviewer/admin acceptance has not yet been completed.
- The running service remains SQLite-backed and single-instance.
- PostgreSQL work is schema and transaction contract only, not a selectable runtime backend.
- Browser OIDC login is not shipped; the browser is a local demo.
- Stdio MCP is disabled in OIDC mode because it cannot bind a verified actor.
- Evidence replay protection is authoritative only inside one SQLite instance.
- Managed key storage, rotation policy, revocation service, external audit anchoring, monitoring, backups, and disaster recovery are not productionized.
- The local Keycloak database and generated CA are development-only.
- No external penetration test, regulated partner assessment, or payment-network certification has occurred.
- No bank, PSP, UPI, wallet, or live merchant integration exists.

11. Roadmap and decision gates

System building should continue, but execution should remain the last component introduced.

Gate 1 - package the next public alpha

Publish the current `main` state as a public research alpha only after hosted checks on the exact release candidate and final documentation review. Include the non-secret component manifest, this paper, source checks, and release notes. Do not publish generated credentials or local runtime reports. This gate does not authorize an external pilot or any payment integration.

Gate 2 - complete real human acceptance before an external pilot

Two real people must complete reviewer and administrator device login, replace temporary passwords, configure TOTP, inspect exact fictional proposals, and make the required decisions. The acceptance record must distinguish human action from automation. No external participant should receive access before this gate passes.

Gate 3 - production foundation

Complete the PostgreSQL runtime port and concurrency parity suite. Add managed keys, durable replay protection, external audit anchoring, observability, backup and restore, migration discipline, tenant authorization, and an independent security assessment.

Gate 4 - controlled partner pilot

Select one fictional or sandbox-only use case with a regulated bank, PSP, or enterprise partner. Agree the responsibility model, liability boundary, authentication, evidence semantics, data handling, support process, and exit criteria before any adapter is implemented.

Gate 5 - separately operated executor

Only a regulated participant should operate the component that converts consumed evidence into a provider instruction. It must live in another trust domain, own credentials and reconciliation, support a kill switch, and refuse anything that cannot be tied to one valid, unconsumed authorization object.

Gate 6 - certification and bounded launch

Complete the relevant provider, security, operational, legal, and regulatory assessments. A technical compatibility result must never be relabelled as NPCI, UPI, bank, or PSP approval.

12. Conclusion

Parimit demonstrates a practical separation between an agent's ability to describe a payment and a system's authority to move money. The current alpha can validate and store proposals, apply deterministic rules, separate machine and human roles, bind decisions to exact intent, issue signed non-dispatchable evidence, and preserve an inspectable audit history.

The local AiNxt compatibility run adds a useful systems result: a reviewed agent runtime and local model can produce a draft that crosses a strict, authenticated sidecar boundary without gaining approval or execution authority. The allowed scenario stopped at [AWAITING_APPROVAL](#); the denied scenario stopped before creation.

That is the achievement and the limit. Parimit is now a tested open-source authorization prototype, not a payment network. Human acceptance, production hardening, regulated partnership, external review, and certification must precede any live payment adapter. The proposal boundary should be preserved as those layers are added.

References

1

Parimit contributors, **Parimit repository**, tested runtime snapshot d9807d3c7f53c25464572f947c8c1daae690c355, September 2026:
<https://github.com/cad07/parimit/tree/d9807d3c7f53c25464572f947c8c1daae690c355>

2

Parimit contributors, **Authorization Envelope v1**: <https://github.com/cad07/parimit/blob/d9807d3c7f53c25464572f947c8c1daae690c355/docs/authorization-envelope-v1.md>

3

Parimit contributors, **Corrected live AiNxt-Keycloak evidence summary**:
<https://github.com/cad07/parimit/blob/main/docs/evidence/live-ainxt-keycloak-smoke-d9807d3.json>

4

NPCI, **AiNxt OS official repository**: <https://github.com/npci/ainxt-os>

5

NPCI, **AiNxt OS control_plane_sha contract**, source-review anchor 454fb09cf1fff2bedb5aa3f4f1c391e4915f33dd: <https://github.com/npci/ainxt-os/blob/454fb09cf1fff2bedb5aa3f4f1c391e4915f33dd/crates/ainxt-protocol/src/lib.rs#L253-L255>

6

OpenID Foundation, **OpenID Connect Core 1.0 incorporating errata set 2**:
https://openid.net/specs/openid-connect-core-1_0.html

7

IETF, **RFC 7515: JSON Web Signature**, May 2015: <https://www.rfc-editor.org/rfc/rfc7515>

8

IETF, **RFC 7517: JSON Web Key**, May 2015: <https://www.rfc-editor.org/rfc/rfc7517>

9

IETF, **RFC 8037: CFRG Elliptic Curve Diffie-Hellman and Signatures in JOSE**, January 2017:
<https://www.rfc-editor.org/rfc/rfc8037>

10

IETF, **RFC 8628: OAuth 2.0 Device Authorization Grant**, August 2019: <https://www.rfc-editor.org/rfc/rfc8628>

11

NIST, **Digital Identity Guidelines: Authentication and Authenticator Management, SP 800-63B-4**, July 2025:
<https://csrc.nist.gov/pubs/sp/800/63/b/4/final>

12

Keycloak, **Server Administration Guide**: https://www.keycloak.org/docs/latest/server_admin/

13

Parimit contributors, **Local AiNxt component manifest v0.2**:
<https://github.com/cad07/parimit/blob/main/docs/evidence/ainxt-local-component-manifest-v0.2.json>

14

Parimit contributors, **Initial live AiNxt-Keycloak feature pull request 9**: <https://github.com/cad07/parimit/pull/9>

Appendix A - Public capability inventory

A.1 Proposal-only MCP tools

Tool	Purpose
create_payment_proposal	Validate, evaluate, and persist an agent-owned proposal
get_payment_status	Read proposal and labelled mock-observation state
cancel_payment_proposal	Cancel an eligible agent-owned proposal
get_policy_decision	Inspect deterministic policy output
simulate_payment	Record or inspect mock behavior only
get_payment_audit	Read local hash-linked audit evidence

No MCP tool can approve, reject, issue or consume evidence, execute, initiate, pay, send, transfer, settle, refund, or retry a payment.

A.2 AiNxt adapter operations

Operation	Authority
parimit_simulate_proposal	Policy simulation only
parimit_create_proposal	Create one agent-owned proposal
parimit_get_proposal	Read the authenticated agent's proposal
parimit_cancel_proposal	Cancel the authenticated agent's eligible proposal

Appendix B - Evaluation boundary

The labels used in this paper have specific meanings:

- **Implemented:** source code and deterministic tests exist in the identified repository snapshot.
- **Automated test passed:** a named automated check reported success.
- **Maintainer-observed live pass:** a local multi-process run reported success and produced a redacted local report.
- **Human acceptance:** requires real people to complete identity and decision steps; this has not yet occurred.
- **External acceptance:** requires an independent organization to review or accept the system; this has not occurred.
- **Payment integration:** requires a regulated provider connection capable of moving or reserving money; this does not exist.

Appendix C - Independent project statement

Parimit is independently implemented. Public specifications and the NPCI AiNxt OS repository were studied to understand the surrounding ecosystem and to separate governance from execution. No claim is made that Parimit is sponsored, reviewed, certified, or endorsed by NPCI, Keycloak, Ollama, the Qwen project, any bank, any payment provider, or any regulator. Names and marks belong to their respective owners.